

real-tr-p8-nohalo

An AgentMPI run analysis

Abstract

This is the parallel book-translation experiment at $p = 8$ with the neighbour exchange removed: eight real agent ranks still agree a global glossary by `allreduce`, but never see how their neighbours rendered the abutting text, so the seams between sections go unreconciled. Against the `real-tr-p8-full` baseline the ablation removes 84 events, 8 of the 24 agent calls, 14 of the 56 messages, 33.3% of prompt tokens, 41.5% of completion tokens, 37.4% of the dollar cost and 24.5% of wall time. The decomposition of that saving is the substance: the halo exchange’s own traffic is 14 messages carrying 361 tokens, 1.14% of the baseline’s tokens sent, and its own blocking is 34.72 rank-seconds, 2.6% of the pool; essentially the entire cost is the second round of agent calls it enables, 381.63 rank-seconds and \$0.0971. The baseline’s much-remarked 20.70 s halo skew turns out to measure nothing about the exchange: each rank’s departure time is exactly the maximum entry time over its closed neighbourhood plus 0.10–0.25 s of fabric latency, verified for all eight ranks, so the exchange is a barrier over the ring graph and the skew is the spread of the preceding translate phase, which the exchange in fact narrows from 25.58 s to 20.70 s. The single thing a reader should take away is negative and it concerns the record rather than the protocol: the boundary-drift cost this ablation exists to measure is not recoverable from what was recorded, because the deterministic scorer has no seam metric *and* because four of the eight sections in this run are byte-identical to artefacts of the baseline — three of them to the baseline’s *post*-halo text. The ablation’s resource arm is sound; its quality arm is contaminated.

1 What this run is

property	value
run	<code>real-tr-p8-nohalo</code>
experiment	translation
campaign label	<code>real-tr</code>
declared world size	8
ranks appearing in the log	10
executors	<code>broker</code> $\times$ 8, <code>worker</code> $\times$ 25
events	263
wall time	125.95 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	$4.46\times$	total busy time over wall time
parallel efficiency	55.7%	achieved parallelism per rank
idle fraction	44.3%	rank-seconds paid for and unused
peak ranks busy	8	of 8 in the world
serial fraction of active time	43.9%	time with exactly one rank busy
load imbalance	$1.61\times$	slowest rank’s busy time over the mean
coordination share	137.4%	wall time inside collectives
rank reattachments	23	fresh agent processes that took over a rank role
agent calls	16	invocations that reached a model
agent latency p50 / p95	42.02 / 49.52 s	per invocation
messages	42	34 eager, 8 rendezvous
tokens sent	31,054	8,083 deferred under rendezvous
tokens in / out	29,000 / 5,054	prompt and completion
cost	\$0.1628	0.0322 \$/kTok out

3 What the analysis notices

- **[warning]** Ranks 8, 14 appear in the log without participating; the declared world size is 8. Shared worker pools let a worker register against the wrong job, which inflates the apparent rank count.
- **[note]** Collectives account for 137.4% of wall time, so this run spent more time coordinating than working.
- **[note]** Load imbalance is $1.61\times$: the slowest rank was busy far longer than the mean.
- **[ok]** 23 rank reattachment(s) occurred, up to incarnation 4: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- **[ok]** All 9 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	67.88	53.9	2	2	9	9	11,957	0.0	finalized
1	61.75	49.1	2	2	3	3	571	0.0	finalized
2	60.24	47.8	2	2	6	6	3,798	0.0	finalized
3	112.82	89.6	2	2	3	3	563	0.0	finalized
4	66.14	52.5	2	2	9	9	8,843	0.0	finalized
5	60.58	48.1	2	2	3	3	676	0.0	finalized
6	64.81	51.5	2	2	6	6	3,998	0.0	finalized
7	67.36	53.6	2	2	3	3	648	0.0	finalized

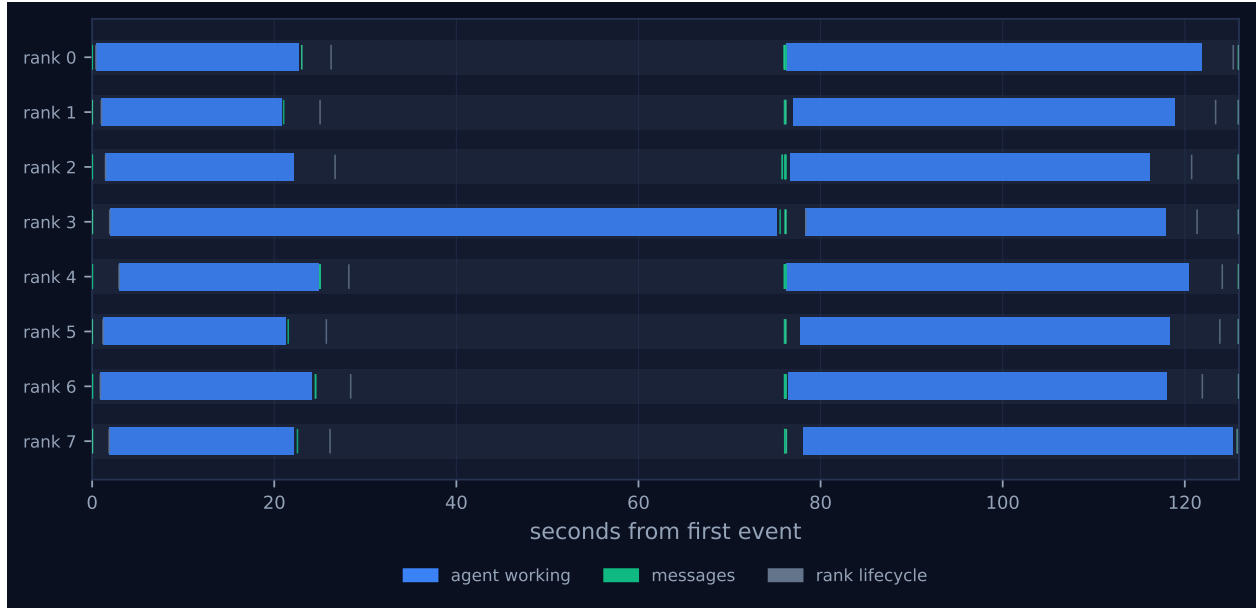


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer’s palette so the same shapes read the same way in both.



Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

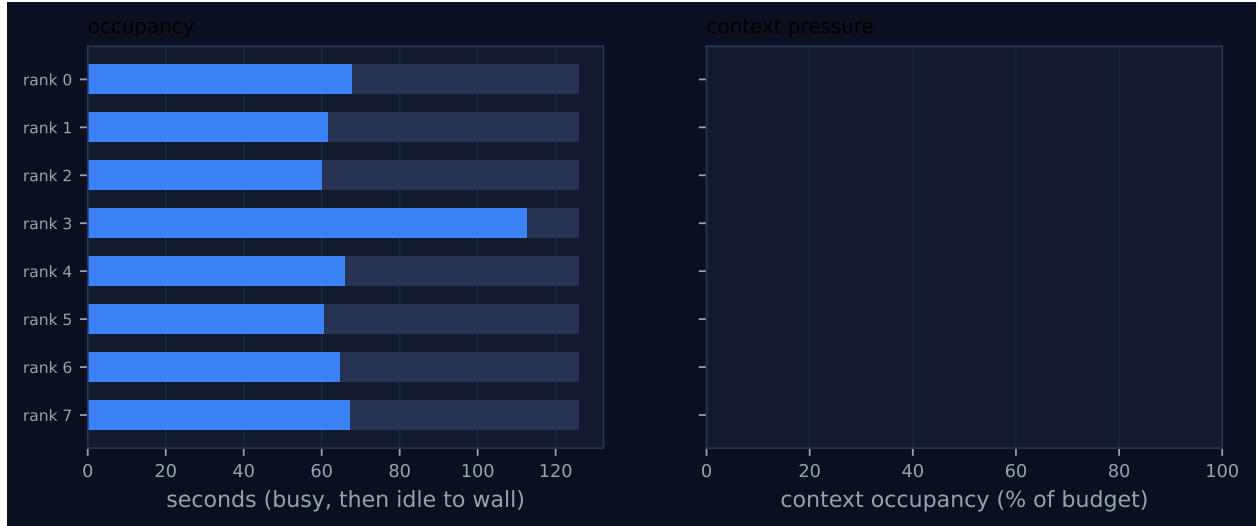


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
scatter	binomial	decompose	8	8	3	3	7	7	11,594	0.03	0.02	✓
reduce	binomial	glossary	8	8	3	3	7	7	2,572	53.27	55.01	✓
bcast	binomial	glossary	8	8	3	3	7	7	11,032	55.02	0.08	✓
allreduce	reduce_bcast	glossary	8	8	6	6	14	14	0	55.02	0.08	✓
reduce	binomial	register	8	8	3	3	7	7	266	0.12	0.11	✓
bcast	binomial	register	8	8	3	3	7	7	1,148	0.18	0.08	✓
allreduce	reduce_bcast	register	8	8	6	6	14	14	0	0.18	0.08	✓
barrier	central	pre-gather	8	8	0	2	0	0	0	9.19	0.16	✓
gather	binomial	collect	8	8	3	3	7	7	4,234	0.08	0.18	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 shows two blocks of work and nothing between them. Every rank has exactly two bars: a term-extraction call beginning within 40 ms of the job start, and a translation call beginning at $t \approx 76$ s. Seven of the eight first bars end between 21.0 and 25.0 s. The eighth, rank 3’s, runs to

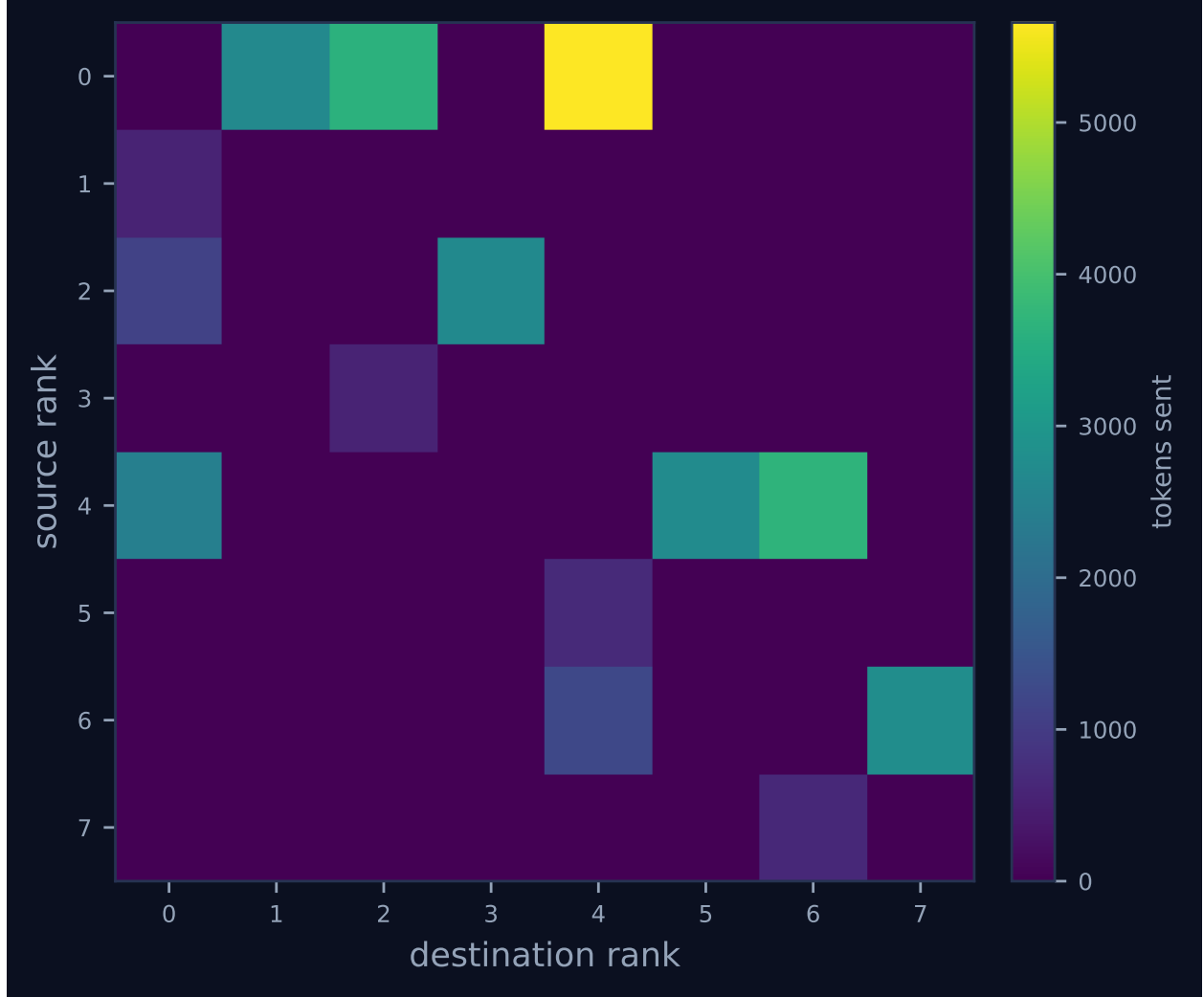


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

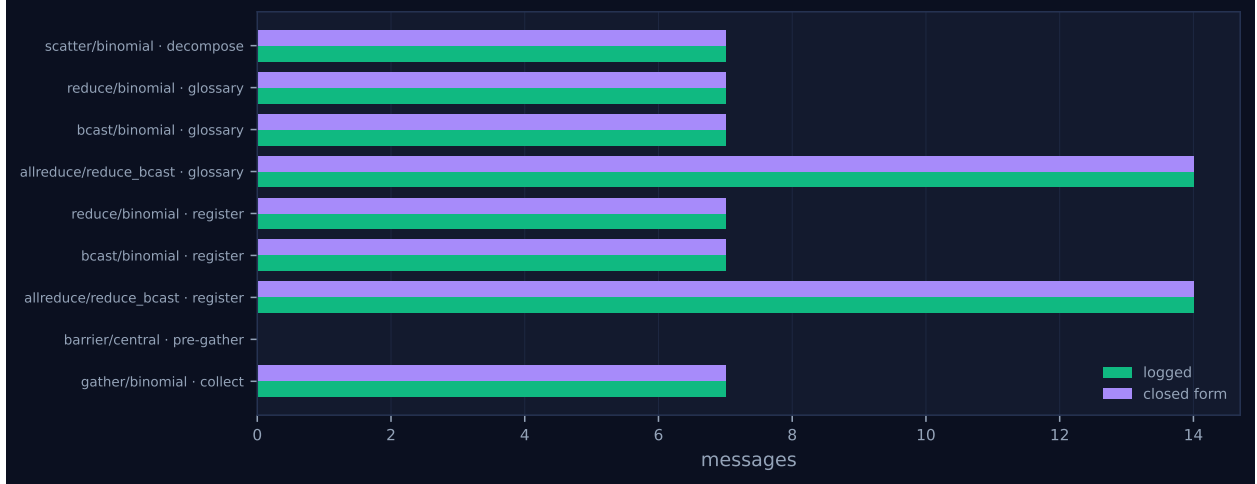


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

75.56 s. From roughly 25 s to 76 s the other seven lanes are empty, and Figure 2 states the same fact numerically: concurrency falls from 8 to 1 shortly after 21 s and sits at 1 for about 50 s before returning to 8 in a single step. That step is what drives the run’s unflattering parallelism figures — 55.7% efficiency, 43.9% of active time with exactly one rank busy, 44.3% of rank-seconds unused — and none of it is attributable to the ablation.

The gap is the glossary `allreduce`, and rank 3 is holding it. Rank 3’s `terms:u3` call reports a latency of 75.53 s against 21.01–25.01 s for the other seven, a mean of 22.86 s; that single call is why rank 3 is busy for 112.82 s at 89.6% occupancy while every other rank is busy for 60.2–67.9 s, and it is the whole of the $1.61\times$ load imbalance. Because the reduction runs as `reduce_bcast` over a binomial tree, no rank can leave until the root has folded every leaf and rebroadcast. The log gives the chain end to end: rank 1 arrives at the reduce at 21.03 s, ranks 5, 7, 6, 4 between 21.53 and 25.03 s, then nothing until rank 3 sends its contribution at 75.56 s, rank 2 folds and forwards at 75.80 s, root 0 emits the completed reduce at 76.04 s, and the broadcast reaches the last rank at 76.13 s. The waits recorded inside that pair are correspondingly large — 53.02 s for the root inside the reduce, 51.02–55.02 s for the six non-root leaves inside the broadcast — and they are the same 55 s of wall clock seen from different sides, a point I return to below because the coordination-share metric does not know it.

The broadcast is a release, and the timeline shows it as one: all eight translation calls start within an 80 ms window, 76.16–76.24 s. They finish between 116.69 and 125.76 s, a 9.0 s spread on latencies of 40.52–49.52 s. The last translation to land, rank 7’s at 125.76 s, is followed within 190 ms by the pre-gather barrier, the binomial gather and job finish at 125.95 s. The barrier is therefore almost free in wall-clock terms even though it absorbed real waiting: the first arrival, rank 2, waited 9.19 s, and the release skew across the eight `coll.barrier` events is 0.16 s. The gather moves 4,234 tokens in 7 messages in 0.19 s.

What is worth looking at in this timeline is mostly what is absent from it. There is no third block. In the baseline the same picture carries a neighbour exchange at $t \approx 81$ –102 s and then a second full round of agent bars from 102 to 166.5 s; here the run ends where the baseline’s translate phase ended. There is no `topo.cart_create` and no `coll.halo_exchange` event, and consequently no nearest-neighbour message ticks. Figure 4 is a clean binomial tree and nothing else: 14 ordered

rank pairs, the seven downward edges $0 \rightarrow 1, 0 \rightarrow 2, 0 \rightarrow 4, 2 \rightarrow 3, 4 \rightarrow 5, 4 \rightarrow 6, 6 \rightarrow 7$ carrying the scatter and the two broadcasts, and their seven reverses carrying the two reductions and the gather. The baseline’s matrix has 20 pairs; the six extra ones are exactly $1 \leftrightarrow 2, 3 \leftrightarrow 4$ and $5 \leftrightarrow 6$, the ring adjacencies the binomial tree does not already contain. Figure 3 shows the imbalance as one long bar among seven short ones and zero context occupancy everywhere, since the harness passes `admit=False` on every rendezvous and 8,083 tokens were deferred rather than admitted. Figure 5 has nine points and no red diamond.

The dashboard screenshot agrees and adds one thing the figures do not. Its collectives panel lists six operations — `reduce`, `bcast`, `allreduce`, `scatter`, `barrier`, `gather` — with no `halo_exchange` row at all, where the baseline’s panel has a seventh row reporting zero messages. The two lanes labelled rank 8 and rank 14 are empty apart from a single grey lifecycle tick each. And the panel shows `allreduce` with 16 invocations and 0 messages, which is not a defect but is the same reading hazard as the halo’s zero: a composed collective’s traffic is attributed to its constituents, so the row that names the operation the harness actually called is the row that appears to have sent nothing.

7 Interpretation

Four things in this run are true by construction, and it is worth listing them so that the rest can be read as evidence. The configuration recorded in the fabric is `"halo": false, "glossary": true`, executor `broker`, 8 ranks, 8 units of 600 words, so the absence of the exchange is the experiment and not a failure. The event deficit against the baseline is exactly accounted for by that absence: $347 - 263 = 84$ events, decomposing as 8 `topo.cart_create`, 8 `coll.halo_exchange`, 28 halo messages (14 sends and 14 receives), 8 `agent.call` for the revision round with their 24 accompanying broker events, and 8 `rank.init` reattachments for the workers those calls would have needed. The message count falls from 56 to 42, and the 14 missing messages are precisely the exchange’s own, $2(p - 1)$ for an open path of eight. And `n_boundary_changes` is 0 for every rank in the result JSON, against 3–5 per rank and 30 in total for the baseline, because the code path that would increment it does not run.

The first thing that emerged is what the baseline’s halo skew actually measures, and the answer is sharp enough to state as a rule. For each rank, take the moment it entered the exchange — its translate call’s completion — and the moment its `coll.halo_exchange` event was emitted, which the log shows is also the moment its revision call began. In the baseline, the departure time of every one of the eight ranks equals the maximum entry time over its closed neighbourhood (itself, left, right) plus a residual of 0.10–0.25 s, mean 0.19 s. Rank 3, which entered at 95.05 s, left at 102.01 s because rank 2 did not enter until 101.88 s. Rank 6, which entered first at 76.30 s, left at 99.02 s because rank 5 entered at 98.79 s. Rank 7 is the only rank that escaped: it sits at the open end of the non-periodic ring with a single neighbour, and that neighbour, rank 6, happened to be the earliest finisher in the run, so rank 7 entered and left at the same timestamp, 81.38 s. The identical relation holds for all eight ranks of `real-tr-p8-noglossary`, which keeps the exchange, with residuals of 0.15–0.25 s.

That makes the exchange a barrier over the ring graph rather than over the world, and at $p = 8$ on a path the distinction buys almost nothing. Seven of the eight ranks in the baseline left within 3.06 s of each other; the 20.70 s reported as skew is one endpoint rank leaving early, once. The corresponding figure in `real-tr-p8-noglossary` is 7.43 s, and there the early leaver is an interior rank whose two neighbours both finished first. So the skew is not a cost of the collective: it is a measurement of the spread of whatever phase precedes it, filtered through a neighbourhood maximum. Indeed the filter

reduces the spread rather than adding to it — the baseline’s translate completions span 25.58s and its halo departures span 20.70s — because clamping every rank up to its neighbourhood maximum compresses the field from below. Removing the exchange, then, did not remove a synchronisation point that was costing this run anything measurable in skew. It removed a phase.

The second thing that emerged is where the saving actually sits, and it is not where the collective is. On the communication side the exchange is nearly free: 14 messages against the run’s 56, and 361 tokens against 31,603 sent, so 1.14% of the fabric’s traffic. Its blocking cost, computed as the sum over ranks of the interval between entering and leaving, is 34.72 rank-seconds out of the $8 \times 166.78 = 1,334.3$ rank-seconds the baseline pool had available, or 2.6%; the figure for `real-tr-p8-noglossary` is 22.97 rank-seconds, 3.0%. What the exchange costs is the round of work it makes possible. The eight `revise` calls consumed 381.63 rank-seconds of agent latency, 28.6% of the baseline pool, with prompts totalling 14,450 tokens and completions 3,661; the observed differences between the runs are $-14,448$ prompt tokens and $-3,582$ completion tokens, matching to within the noise of the term sheets. In dollars the ablation costs \$0.1628 against \$0.2599, a saving of \$0.0971 or 37.4%. This is exactly the shape the project’s argument about halo exchange predicts — $\Theta(p)$ coordination, with the expense in local work — and it is worth noting that the arithmetic alternative is $p(p-1) = 56$ point-to-point messages for an all-gather of boundaries against the exchange’s 14, a ratio of $p/2$ that grows with the world size.

Wall time needs more care than the headline difference suggests. The runs differ by 40.84s, 24.5%, but that number is contaminated in both directions. This run lost about 50.5s to rank 3’s extraction straggler, which has nothing to do with the halo, and gained about 21.8s on a faster translate phase (49.52s against 71.53s for the slowest call). The clean measurement is internal to the baseline: its last translation landed at 101.88s and the job finished at 166.78s, so everything downstream of translation — the neighbourhood wait, the revision round, the barrier and the gather — occupied 64.90s, 38.9% of that run’s wall clock, of which the exchange’s own wait was 0.20s. Remove the exchange and you remove essentially all of that tail, which is what this run shows: 190ms elapse between its last translation and job finish.

The third thing is the instrumentation gap, and this run’s contribution to it is a warning about reading metrics as verdicts. The baseline’s eight `coll.halo_exchange` payloads contain three fields — `label`, `left`, `right` — and nothing else: no `messages_sent`, no `tokens_sent`, no `rounds`, no `algorithm`, no `size`, and no `wall_s`, which is why the collective table records the exchange with algorithm “?”, zero messages and a wall time of 0.0 while the fabric independently logged 14 messages and 361 tokens against its internal tag. The collective whose entire justification is a lower message count was the one collective not reporting its message count. It is fully recoverable from the log — the tags `_mpi:halo:0:8:R` and `_mpi:halo:0:8:L` carry 14 `msg.send/msg.recv` pairs, matching $2(p-1)$ exactly — which is how the per-invocation `logged_messages` field caught the disagreement in the first place. In this run, `n_misreported_collectives` is 0 and the generated findings report that all nine checkable collectives agree with their cost model. Both statements are true and neither means anything: the only collective that disagreed was deleted, so the metric improved by removing the subject rather than by fixing it. The defect itself is real but no longer live — the traces were recorded at commit `b0b652d` and the omission was repaired in `ee21d79`, whose message notes the reason it survived, namely that the existing test asserted on a runtime counter rather than on what the collective wrote to the log. Under the contract’s instruction not to regenerate traces, this run’s log cannot demonstrate the fix; it can only show the hole where the collective was.

The fourth thing that emerged is a check on the glossary machinery that the ablation was not

designed to provide. The resolved glossary in this run has the digest 0233d780cdb6 and 206 terms at every one of the eight ranks, and the baseline’s has the same digest and the same count — even though the two runs’ local term sheets differ (24–38 terms per rank in the baseline against 27–39 here) and their conflict counts differ (4 against 2). Different contributions, different conflicts, identical agreed result at every rank: that is the claim the harness makes for UNION with deterministic conflict resolution, and it holds here across two independent extraction phases.

Which brings the analysis to the question the ablation was actually for, and to a negative answer. The result JSON for this configuration exists at `results/translation/real-tr-p8-gloss-nohalo-union-reduce_bca` and its quality block is indistinguishable from the baseline’s: coverage 1.0, consistency 1.0, six shared entities, zero divergent entities, 22 renderings all verified, paragraph fidelity 1.0, length plausibility 1.0, target-script ratio 0.8578 against the baseline’s 0.8579, residual source per kilo-character 0.296 against 0.297. This is not evidence that the seams survived. It is evidence that the scorer cannot see seams: reading `score_translation`, every metric it computes is either per-unit (paragraph count, length ratio, script ratio, residual source) or about entity agreement across units, and none of them examines the junction between one unit’s last paragraph and the next unit’s first. The one quantity that would have moved, `n_divergent_entities`, is zero even in `real-tr-p8-noglossary`, so it is not discriminating at this scale either.

What the record does allow is a measurement of what the exchange produced in the baseline, recovered from that run’s broker spool, where each revision prompt embeds the pre-revision text verbatim between its `YOUR CURRENT TRANSLATION` markers. Comparing that against the revision’s output for all eight sections: the revision changed every section, and changed 70 of 9,527 characters, 0.73% overall and between 0.25% and 2.24% per section. The agents’ own `changes` lists, 30 entries across the eight, describe exactly the failures the exchange was introduced to fix — a first paragraph rewritten so that it no longer reads as the opening of a document, a closing sentence extended so that it hands over to the next section, a second-person pronoun unified with the neighbouring section’s usage, and one terminology fix that disambiguates two glossary entries for carriage drivers that the glossary itself could not resolve without seeing the neighbour’s text. So the exchange’s product is real, visible in the artefacts, and worth 0.73% of the characters for 38.9% of the wall clock. Whether losing it degrades the book is a question this trace cannot answer, and the honest statement is that the ablation’s quality arm did not measure anything — not that the seams were fine.

Read against its siblings, this run is the resource-side half of a pair. `real-tr-p8-noglossary` removes the `allreduce` and keeps the exchange, so it drops the coordination and keeps the second round of local work; this run does the reverse. The two are not symmetric in cost: at 204 events and \$0.1355, removing the glossary is cheaper than removing the halo (263 events, \$0.1628) against the baseline’s 347 and \$0.2599, because the glossary’s cost is mostly the extraction round and the broadcast of a 206-term artefact, while the halo’s is a full re-translation. Taken together they say that at $p = 8$ the expensive part of the HPC decomposition is never the collective.

The remaining flagged findings resolve as follows. Ranks 8 and 14 appearing without participating is expected and not specific to this configuration: all three siblings show the same two stray identifiers with a single lifecycle event each, which is a shared worker pool registering against the wrong rank slot, and no message or collective in the log involves them. The 23 reattachments up to incarnation 4 are also expected, being two agent calls per rank each served by a fresh worker process. The $1.61\times$ imbalance is a genuine property of this run but not of this ablation; it is rank 3’s extraction call and nothing else.

The 137.4% coordination share is a metric defect and should not be read as a statement about the

run. A collective’s `wall_s` in this analysis is the maximum per-rank blocking time inside it, not a duration on the wall clock, and `collective_wall_s` sums those maxima over all invocations. That double-counts twice over: the composed `allreduce` is summed alongside the `reduce` and `bcast` it delegates to, and, worse, the 53.27s that rank 2 spent waiting inside the reduce and the 55.02s that rank 1 spent waiting inside the broadcast are concurrent waits by different ranks in the same 55s of wall clock. Excluding the composed operations still leaves 93.6%, so the composition is not the main problem. The defensible figure is per-rank: summing each rank’s blocking time across the leaf collectives gives 421.35 rank-seconds against 1,007.6 available, 41.8%, and adding the 585.72 rank-seconds of agent latency accounts for 99.9% of the pool. The same computation on the baseline gives 219.25 rank-seconds in leaf collectives plus 34.72 in the halo, 19.0% in total, with 99.9% again accounted for. So the ablation *raised* the coordination share from 19.0% to 41.8%, and it did so by removing work rather than waiting: the denominator of useful activity shrank while the one large wait, the glossary reduction dilated by rank 3, stayed. That is a real and slightly counter-intuitive result about ablations, and it is invisible in the reported 35.7%-to-137.4% comparison, which is not a comparison of shares at all.

8 Threats to this reading

The most serious threat is specific and it is fatal to one arm of the experiment: the translations in this run are not statistically independent of the baseline’s, and in several sections they are not independent in the strongest possible sense. Recovering both runs’ translate outputs from their broker spools and hashing them, four of the eight sections here are byte-identical to an artefact of the baseline. Sections 0, 4 and 5 are byte-identical to the baseline’s *post*-halo text — that is, this run’s supposedly unreconciled output already contains the baseline’s seam repairs — and section 3 is byte-identical to the baseline’s pre-halo text. Section 7 reproduces all six of the multi-character fragments its baseline revision introduced and then adds one further edit of its own; across all eight sections, 10 of the 14 multi-character fragments the baseline’s revision introduced appear verbatim here. The translate prompts are not identical between the runs (they differ by one character on the shared-register line), so a prompt-digest cache cannot explain this, and no independent sampling reproduces 1,371 characters of Chinese prose byte for byte. The two runs were recorded 3 min 41 s apart on the same commit in the same working tree, with the baseline’s `runs/real-tr-p8-full/output/` and spool on disk, and the `broker` executor works by publishing prompts to a spool for an external worker to fill in. The straightforward explanation is that the worker reused the earlier run’s text. This does not affect any of the protocol measurements — message counts, event structure, timings and the neighbourhood-maximum result are all properties of the harness and the fabric, not of the text — but it means the boundary-drift comparison cannot be made from these two runs even if a seam metric existed, because five of eight sections already carry the baseline’s boundary repairs. It also retracts the strength of the identical-glossary observation above: a run that reuses the neighbour’s outputs is not the clean A/B that identical digest would otherwise imply.

The executor is `broker`, so this is not a surrogate standing in for a model, but the way the broker is driven leaves two marks on the numbers. Agent latencies are quantised to `BrokerExecutor.poll_s`, 0.5s, which is visible in the log as every latency landing on a half-second plus 10–30 ms; the reported p50 and p95 of 42.02 and 49.52s are therefore accurate only to about half a second, which does not matter at these magnitudes but does mean the latency distribution is not the model’s. And the provenance block records `tokenizer_exact: false`, so every token figure is a heuristic estimate.

That is visible: a 1,371-character Simplified Chinese section is reported as roughly 400 completion tokens, which is low by a factor of two or more. The token deltas I quote — 14,448 prompt and 3,582 completion — are consistent with the revision calls’ own reported totals and so are internally coherent, but their absolute scale, and therefore the \$0.0971 saving, inherits the tokenizer’s error.

Rank 3’s 75.53s extraction call is one draw from one run, and it is doing most of the work in every aggregate that looks bad here: the 55s dead zone, the $1.61\times$ imbalance, the 55.7% parallel efficiency and the 41.8% coordination share are all downstream of it. If it had matched the slowest of the other seven this run would have finished near 75s rather than 126s, which would have made the ablation look considerably better than the 24.5% wall-time saving I report and would have inverted the coordination-share comparison. I have kept the reported numbers and separated the straggler’s contribution where it matters, but no single run can distinguish a straggler that is a property of the configuration from one that is a property of the afternoon.

The neighbourhood-maximum result is the strongest claim in this document and it is verified on 16 rank-instances across two runs, all at $p = 8$ on a one-dimensional non-periodic ring. It should not be extrapolated without checking. On a periodic ring no rank has a single neighbour and the endpoint escape that produces the 20.70s figure cannot occur; in two dimensions the neighbourhood is four ranks and the maximum is taken over more of the population, so the exchange moves further towards a true barrier. The conclusion that a halo exchange is “a synchronisation point in all but name” is therefore conservative for larger neighbourhoods and specific to open boundaries for the escape.

Finally, two of my comparisons are arithmetic rather than measurement, and should be read that way. The $\Theta(p)$ -versus- $\Theta(p^2)$ contrast — 14 messages against 56 — compares the exchange’s logged traffic to the message count an all-gather of boundaries would require; no sibling run performed that all-gather, so the alternative is computed from the topology and not observed. And the claim that the scorer is blind to seams is inferred from reading its metric definitions together with the identical scores; I did not implement a boundary-continuity metric and run it over the two runs’ outputs, which is what would settle the question. Given the contamination described above, doing so on these particular artefacts would not settle it either.